MATLAB 완벽가이드



저자 : 임종수

이 문서는 한글97로 제작되었습니다. 하지만 도스용 한글3.0부터 윈도우용 한글97까지 완벽히 지원됩니다. 만약 그림부분이 나타나지 않는다면 RAM이 얼만큼인지 보시기 바랍니다. RAM용량이 적으면 나타나지 않을 수 있습니다. 또한 이 글을 보시게 될 분의 글꼴이 어떤지 모르기에 가장 기본이 되는 글꼴만 이용했음을 미리 알려드립니다.
기타 문의 사항이나 더욱더 알고 싶으신분은 다음의 연락처 혹은 홈페이지에 방문하시기 바랍니다.



하이텔 : ssljk07
E-mail : matlab@loveangel.net

제작·편집 : 임종관 「오정, 한글서당, MATLAB 전문학교, 이야기세상 운영자」

http://matlabschool.com [MATLAB 전문학교]

10월 6, 7일 이틀에 걸쳐...
임종수씨가 진행하는 세미나 개최!!

▶ 날짜와 안내

수강 인원 : 100명
세미나 날짜 : 2001년 10월 6일 (토) ~ 10월 7일 (일)
세미나 장소 : 서울 대학로 정보세미나홀 (자세한 안내도는 홈페이지에 기재)
세미나 시간 : 오전 10시 ~ 오후 6시 (12시 ~ 1시는 점심시간, 음료수 제공)
세미나 교재 : 임종수의 MATLAB 완벽가이드
(당일 시중가의 80%에 판매 - 시중가 18,000원 → 판매가 14,400원)
세미나 방식 : 5시간 강의후 2시간 질문/답변 시간

세미나 수강료 : 학생(하루 4만원, 이틀 7만원), 일반(하루 5만원, 이틀 9만원)
수강료 납부방식 : 선불이며, 무통장입금과 신용카드결제 두가지 방식으로 진행합니다.
수강신청 : http://matlabschool.com/seminar.html

▶ 강의 내용

첫째날(10월 6일 토요일)

1. matlab overview
2. matlab의 행렬 연산.
3. M file 작성 방법.
4. Matlab 그래픽스.
5. 파일입출력과 GUI programming 기본

둘째날(10월 7일 일요일)

1. Handle Graphics system.
2. Cell and struct data type.
3. 다양한 GUI Programming 방법.

※ 이번 강의는 MATLAB 5.x에 대한 것이며, 다음에는 6.x에 대한 강의를 진행할 예정입니다.
※ 이번 강의를 수강한 모든 분들께는 다음 강의를 수강료를 할인해 드립니다.



라) contour3.m : 제 1 부류

3차원 contour plot을 작성해주는 함수이다.
앞에서 배운 "contour.m"함수와 사용 방법이 비슷하다.

전형적인 사용방법(syntax) : **contour3.m**

a) [C, h]=contour3(Z)
***행렬 Z는 x-y 평면대한 높이로 해석된다.**
Contour **행렬 C와** Line object**에 해당하는**
handle value, **h를 반환한다.**
***"clabel.m"함수를 이용하여 각각의 등고선에 label을 붙인다.**

b) [C, h]=contour3(Z, n)
***행렬 Z에 대한 n개의 등고선을 그린다.**

c) [C, h]=contour3(Z, v)
***행렬 Z에 대한 등고선을 벡터 v의 값에 근거하여 그려 준다.**

d) [C, h]=contour3(X, Y, Z), [C, h]=contour3(X, Y, Z, n(or, v))
***행렬 X, Y는 x, y축의 범위를 정한다.**

e) [C, h]=contour3(...., LineSpec)
***LineSpec에 근거하여 각각의 등고선들을 그린다.**

"**LineSpec**"은 line**의** style**을 결정하는 지정어들의 조합이다.**
다음에 나오는 "stem3.m"함수 설명부분에 있는 "LineSpec" 용어 설명부분을 참고하기 바란다.

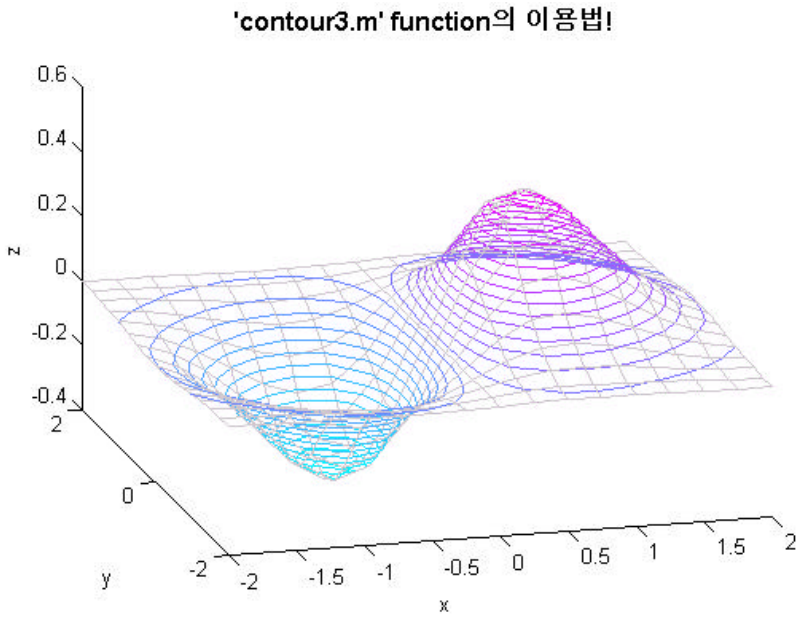
```
[X,Y]=meshgrid([-2:.25:2]);
Z=X.*exp(-X.^2-Y.^2);
contour3(X,Y,Z,30)
hold on
h=surf(X,Y,Z);
set(h,'edgecolor',[.8 .8 .8],'facecolor','none')
hold off
grid off
view(-15,25)          %방위각:-15, 고도:25
colormap cool
set(gcf,'color','w')
xlabel('x');
ylabel('y');
zlabel('z');
title('\bf{''contour3.m'' function의 이용법!}','fontsize',12)
```

위와 같은 일련의 명령어를 실행하면, [그림 3-55]**와 같은 결과를 얻는다.**

앞에서 언급하였듯이, "**surf.m**"**함수는 surface object에 해당하는** handle**을 돌려준다.**

Surface Object	
propertyname	propertyname에 대한 설명
EdgeColor	Surface를 구성하는 각각의 patch에 대한 모서리의 색을 결정한다. default는 검은 색이다.
FaceColor	Surface를 구성하는 각각의 patch에 대한 표면의 색을 결정한다.

그러므로, "□set(h,'edgecolor',[.8 .8 .8],'facecolor','none')" **명령에서** "edgecolor"**가** [.8 .8 .8]**인 것은 모서리 색을** [.8 .8 .8]**로 하고**, "facecolor"**가** "none"**이라는 것은 표면을 그리지 말라는 뜻이다**.



[그림 3-55] "contour3.m"**함수의 사용 방법**

마) contourf.m : 제 1 부류

"**contourf.m**"함수는 2차원 contour plot을 그려주는 데, 등고선들 사이의 면적들(areas)에 일정한 색을 첨가해 준다. 이들 색은 현재 적용되고 있는 **colormap**에 의해 결정된다.

전형적인 사용방법(syntax) : **contourf.m**

a) [C, h, CF]=contourf(Z)
→행렬 Z는 x-y 평면대한 높이로 해석된다.
→Contour 행렬 C와 Line Object에 해당하는 handle value, h를 반환한다.
→CF는 주어진 "colormap"에 의해 채워진 면적들에 대한
→contour matrix이다.
→"**clabel.m**"함수를 이용하여 각각의 등고선에 label을 붙일 수 있다.

b) [C, h, CF]=contourf(Z, n)
→행렬 Z에 대한 n개의 등고선을 그린다.

c) [C, h, CF]=contourf(Z, v)
→행렬 Z에 대한 등고선을 벡터 v의 값에 근거하여 그려 준다.

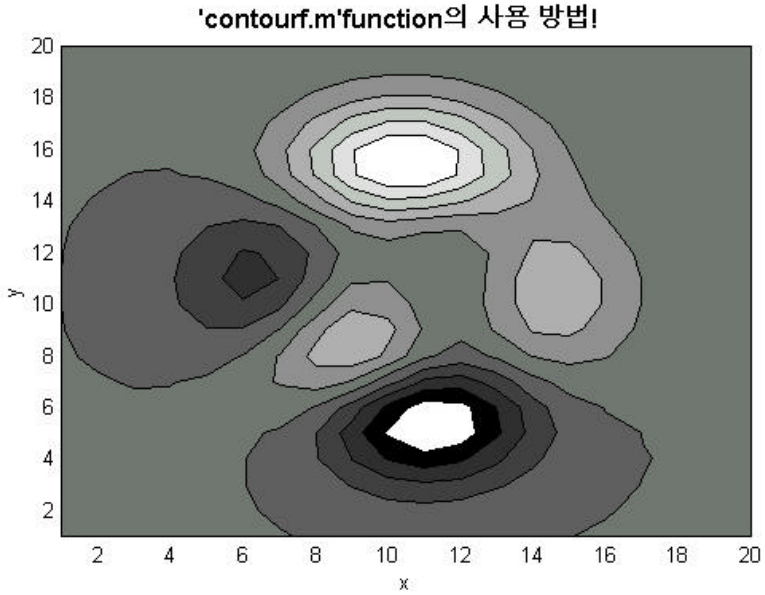
d) [C, h, CF]=contourf(X, Y, Z), [C, h]=contourf(X, Y, Z, n(or, v))
→행렬 X, Y, Z는 x, y, z축의 범위를 정한다.

다음과 같은 예제를 생각해보자.

```
[C,H,CF]=contourf(peaks(20),10);
colormap gray
xlabel('x')
ylabel('y')
title('\bf{''contourf.m''function의 사용 방법!}','fontsize',12)
set(gcf,'color','w')
```

위와 같은 일련의 명령을 실행하면, colormap "**gray**"에 의거하여 등고선들 사이에 지정된 색을 채워준다.

[그림 3-56]은 그 결과를 보여 주고 있다.



[그림 3-56] "contourf.m"함수의 사용 방법!

바) stem.m : **제 1 부류**

모든 불연속 데이터의 열(discrete sequence data)**을 표현하는데 이용되는 함수이다.**

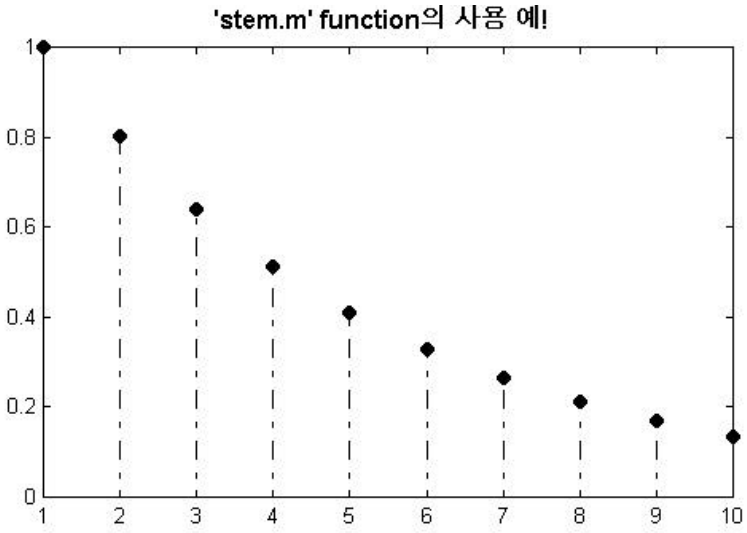
전형적인 사용방법(syntax) :**stem.m**

h=stem(x,y)
→**벡터** y**를 각각의 벡터** x**에 대해서 표현 해준다.**
→**Line Object에 해당하는** handle, **h를 돌려준다.**
h=stem(x,y,'fill')
→**각각의** marker**에 색을 채워준다.**
h=stem(..., LineSpec)
→stem plot**에 대한 선,** marker, color**의** style**을 결정 해준다.**

사실, "stem.m"함수는 디지털 신호처리 분야에서 데이터를 표현하는데 많이 이용되는 함수이다.

```
□y=linspace(0, 2, 10);     % 0     2사이를 균등하게 10개의 데이터로 나눔!
□stem(exp(-y), 'fill', '-.')
□axis([0 11 0 1])
```

위와 같은 일련의 명령어를 실행하면 [그림 3-57]**을 얻는다.**



[그림 3-57] "stem.m" function의 사용 방법

사) stem3.m : 제 1 부류

"**stem.m**"함수에 대한 3차원 표현 방법이다.

전형적인 사용방법(syntax) :**stem3.m**

a) **h=stem3(x,y,z)**

***벡터 z를 각각의 벡터 x, y에 대해서 표현 해준다.**

***Line Object에 해당하는 handle, h를 돌려준다.**

b) **h=stem3(x,y,z,'fill')**

***각각의 marker에 색을 채워준다.**

c) **h=stem3(..., LineSpec)**

*stem3 plot**에 대한 선**, marker, color**의** style**을 결정 해준다.**

다음과 같은 예제를 통하여 "stem3.m"함수의 사용 방법을 알아두자.

[**예 제**] **임의의 데이터를** FFT(Fast Fourier Transform)**한 결과는 복소 평면상의 단위 원**(unit circle)**상에 분포하는** magnitude**로 표현된다. 그러므로, 주어진 데이터에 대한** FFT**의 결과가 단위 원주 상에서 어떻게 분포하는지 알아보는 것은 매우 흥미로운 일이 될 것이다.**
step function의 주파수 응답에 대한 magnitude가 단위원주 상에서 어떻게 분포하는지 조사하여라.

[**풀이**]
우선, 단위 원을 만들기 위한 데이터를 구성한다.

```
th=(0:127)/128*2*pi;          %여기서는 128 point의 데이터를 취했다.
x=cos(th);
y=sin(th);
```

이제, step function**의 주파수 응답에 대한** magnitude**를 계산 해보자.**

```
f=abs(fft(ones(10,1),128));          %step function을 128-point FFT하여,
                                     %그 결과에 대한 magnitude를 계산.
stem3(x, y, f','d','fill');
view([-65 30])              %데이터의 분포를 보다 명확히 보기 위해서
                            %관측 점의 위치를 바꾸어 보았다.
xlabel('\bf{Real}','fontsize',12);
ylabel('\bf{Imaginary}','fontsize',12);

zlabel('\bf{Amplitude}','fontsize',12);
title('\bf{Magnitude Frequency Response}','fontsize',12)
```

"**stem3.m**"**함수에서 입력 매개변수로 사용된 'd'는** marker type**을 diamond로 하라는 뜻이다.**

[표 3-11]은 **marker symbol**의 종류와 각각의 marker를 지정하는 방법에 대해서 요약한 것이다.

그런데, 이들 지정어(specifier)들은 모두 string이므로, 단일 인용부호(즉, ' ')로 둘러싸야 한다.

보통, Line Object에 해당하는 handle을 돌려주는 함수들은 line에 대한 color([표3-9]참고), marker symbol([표3-11]), line type([표3-12])의 지정어들을 조합하여 line의 style을 결정한다. 이들 line의 style을 결정하는 지정어들의 조합을 보통, "**LineSpec**"이라고 한다.

Marker Specifier	Description
+	plus sign
o	circle
*	asterisk
.	point
x	cross
square	square
diamond	diamond
^	upward pointing triangle
v	downward pointing triangle
>	right pointing triangle
<	left pointing triangle
pentagram	five-pointed star
hexagram	six-pointed star

[표 3-11] "marker" symbols의 종류

이들 marker symbols의 지정어(specifier)들에 대한 약식 표기는 머릿글자이다.

그러므로, "**diamond**"대신에 "**d**", "**hexagram**"대신에 "**h**"라고 하여도 된다.

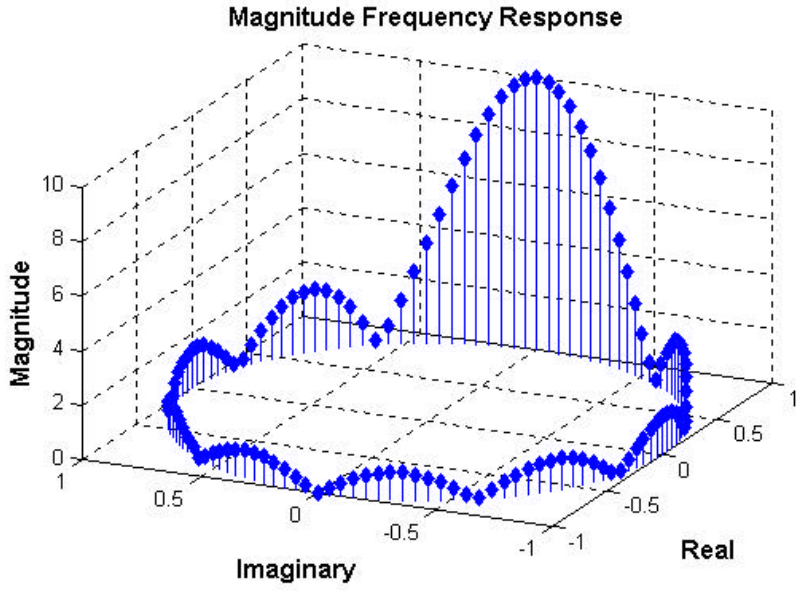
[표3-12]는 Line type을 지정하는 방법과 종류가 요약되어 있다.

Symbol	Line Style	Symbol	Line Style
-	solid line (default)	:	dotted line
--	dashed line	-.	dash-dot line

[표 3-12] "Line" type의 종류와 지정 방법

결국, 이들 [표 3-9], [표3-11], [표3-12]는 모든 Line Object에 해당하는 handle을 돌려주는 함수들에서 Line style을 결정하는데 사용할 수 있다.

"stem3.m"함수를 이용하여 계산된 magnitude는 [그림 3-58]에서 볼 수 있다.



[그림 3-58] "stem3.m"함수의 사용 방법

[그림 3-58]로부터 step function을 FFT한 결과는 sinc function의 형태를 취한다

아) stairs.m : 제 1 부류

"**stairs.m**"함수는 sampling 된 신호에 대한 time-history를 작성하는데 유용한 함수이다. 즉, 불연속 데이터를 "**sample-and-hold**"회로에 통과시킨 결과를 작성하는데 필요한 함수이다.

전형적인 사용 방법 : stairs.m

a) **h=stairs(Y)**
→Y가 벡터인 경우의 X축 범위는 1 length(Y)이고,
→Y가 행렬인 경우에는 각각의 열 벡터가 하나의 신호를 표현하며,
→x 축의 범위는 1부터 행렬 Y의 행의 수가된다.
→Line Object에 해당하는 handle, **h**를 반환한다.

b) **h=stairs(X, Y)**
→배열 Y에 대한 각각의 열 벡터를 배열 X에 대해 표현한다.

c) **stairs(..., LineSpec)**
→stairs 그래프를 구성하는 선(Line)들의 style을 결정한다.

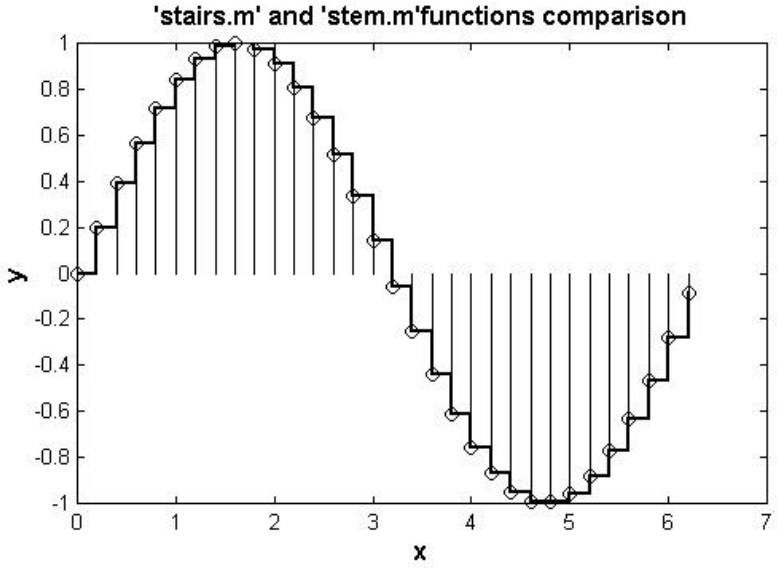
d) **[xb, yb]=stairs(Y), [xb, yb]=stairs(X, Y)**
→그래프를 만드는 대신에 벡터 xb와 yb를 반환하여
→다른 그래픽 함수에서 이용 할 수 있도록 한다.
→예 : plot(xb, yb)

앞에서도 언급하였듯이 하나의 배열로 구성된 신호들의 집합은 주어진 배열에 대해 각각의 열 벡터로 표현된다. 결국, 주어진 배열이 m개의 열을 가지고 있다면, 그 배열은 종류가 m개인 신호데이터를 갖고 있다고 볼 수 있다.
Matlab은 거의 대부분의 함수들이 **column-wise** 연산을 행한다는 데 주의하자!

[그림 3-59]는 "**stem.m**"함수와 "**stairs.m**"함수를 동시에 표현한 것이다.
"**stem.m**"함수가 데이터의 sampling 결과라면, "**stairs.m**"함수는 sampling 된 신호를 "**Sample-and-Hold**"회로에 통과시킨 결과를 나타낸다.

다음과 같은 일련의 명령어들은 [그림 3-59]를 생성 해준다.

```
x=0:1/5:2*pi;
y=sin(x);
stem(x,y,'k')
set(gca,'nextplot','add')                %Axes Object 내용 참고.
h=stairs(x,y,'k');
set(h,'linewidth',2)
set(gcf,'color','w')
xlabel('\bf{x}','fontsize',12)
ylabel('\bf{y}','fontsize',12)
title('\bf{''stairs.m'' and ''stem.m''functions comparison}','fontsize',12)
```



[그림 3-59] "stem.m and stairs.m"함수들의 비교

자) scatter.m : 제 1 부류

"scatter.m, scatter3.m"함수는 2, 3차원 평면상에 대해 벡터 X, Y(3차원인 경우에는 Z가 첨가됨)로 지정된 위치에 colored markers를 표현해주는 함수이다. 그러므로, 분자 구조, 유전자의 구조를 그래프화 하는 데 매우 유용한 함수이다.

전형적인 사용 방법(synatx) : **scatter.m**

a) **h=scatter(X, Y, S, C)**
*S가 **scalar인 경우에는 모든** marker size**를** S**크기로 같게 한다.**
*S가 **벡터인 경우에는 벡터** X, Y**와 크기가 같아야 하고,**
모든 marker size**가 벡터** S**에 의존한다.**
*C가 **벡터인 경우에는 벡터** X, Y**와 크기가 같아야 하고,**
벡터 C**의 원소들을 현재 적용되는** Colormap**에 선형으로**
mapping**시켜서** marker color**를 결정한다.**
*C가 length(X)×3**의 크기를 갖는 행렬이면,** markers**의** color**를 새로 생긴** colormap**인** lebgth(X)×3**의 값에 의해서 결정한다.**

b) **h=scatter(X, Y)**
*주어진 벡터 X, Y**에 의해서 결정된 위치에** marker**를**
default color**와** size**로 위치시킨다.**
*Line Object**에 해당하는** handle, **h를 반환한다.**

c) **h=scatter(...,*markertype*)**
*marker**의** type**을 지정해 줄 수 있다.**

d) **h=scatter(..., 'filled')**
*markers**의 표면을 주어진 색으로 채워 준다.**

예제를 통하여 보다 자세히 "scatter.m"함수에 대해서 알아보자.

[예 제] 반지름이 1인 구를 "sphere.m"함수로 만든다. 만들어진 구를 여러 높이에서 수평으로 자른 경우에 해당하는 2차원 scatter그래프를 만들어 보자.

[풀이]

전형적인 사용 방법(syntax) : **sphere.m**

a) **[X, Y, Z] = sphere(n)**

*3개의 (n+1)×(n+1) 크기를 갖는 행렬 X, Y, Z를 만든다.

만들어진 행렬 X, Y, Z로 surf(X, Y, Z)는 반지름이 1인 구를 만든다.

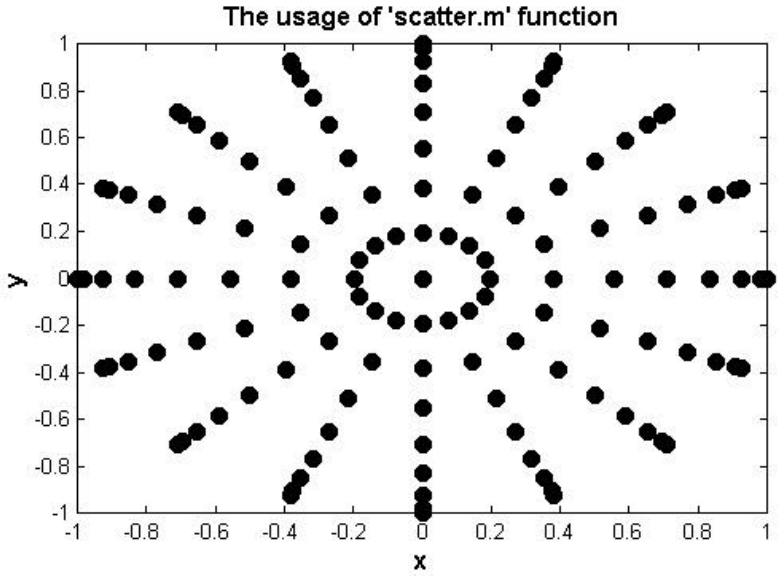
b) **[X, Y, Z]=sphere**

*21×21 크기를 갖는 행렬 X, Y, Z를 만든다.

```
□[x, y, z]=sphere(16);
□xx=x(:);                          %"xx=x(:);"의 의미는 앞에서 설명하였다.
□yy=y(:);
□S=repmat(8,prod(size(x)),1);      %"repmat.m"function은 앞에서 설명하였다.
□scatter(xx(:),yy(:),S(:),'filled')
□scatter(xx(:),yy(:),S(:),'k','filled')
□set(gcf,'color','w')
□xlabel('\bf{x}','fontsize',12)
□ylabel('\bf{y}','fontsize',12)
□title('\bf{The usage of ''scatter.m'' function}','fontsize',12)
□set(gca,'box','on')
```

Axes Object	
propertyname	propertyvalue
Box	"**on**"인 경우에는 active figure에 Box를 첨가한다. "**off**"인 경우에는 Box를 지운다.

위와 같은 일련의 명령어를 실행하면 [그림 3-60]을 얻는다.



[그림 3-60] "scatter.m"함수의 이용법

차) scatter3.m : **제 1 부류**

"**scatter.m**"함수의 3차원적 표현 방법이다. 사용방법은 "**scatter.m**"함수와 거의 같다.

전형적인 사용 방법(synatx) : **scatter3.m**

a) **h=scatter(X, Y, Z, S, C)**
*S가 **scalar**인 경우에는 모든 marker size를 S크기로 같게 한다.
*S가 벡터인 경우에는 벡터 X, Y, Z와 크기가 같아야 하고,
모든 marker size가 벡터 S에 의존한다.
*C가 벡터인 경우에는 벡터 X, Y, Z와 크기가 같아야 하고,
벡터 C의 원소들을 현재 적용되는 Colormap에 선형으로
mapping시켜서 marker color를 결정한다.
*C가 length(X)×3의 크기를 갖는 행렬이면, markers의 color를 새로 생긴 **colormap**인 length(X)×3의 값에 의해서 결정한다.

b) **h=scatter3(X, Y, Z)**
*주어진 벡터 X, Y, Z에 의해서 결정된 위치에 marker를
default color와 size로 위치시킨다.
*Line Object에 해당하는 handle, **h**를 반환한다.

c) **h=scatter3(...,*markertype*)**
*marker의 type을 지정해 줄 수 있다.

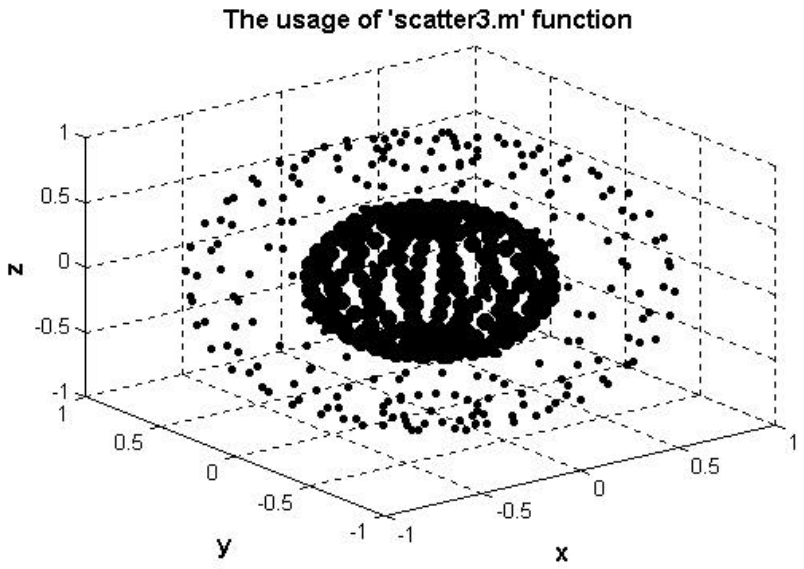
d) **h=scatter3(..., 'filled')**
*markers의 표면을 주어진 색으로 채워 준다.

[예 제] 앞에서 살펴본 [예 제]에서는 3차원 구를 여러 높이에 자른 결과를 알아보았다. 여기서는 반지름이 1인 구안에 반지름이 0.5인 구를 포함시킨 scatter그래프를 작성해보겠다.

[풀이]
먼저, 반지름이 1인 구를 만든 뒤에 0.5를 곱하여 반지름이 0.5인 구를 만들어보자.

```
[x, y, z]=sphere(16);
xx=[x(:)*.5 x(:)];
yy=[y(:)*.5 y(:)];
zz=[z(:)*.5 z(:)];
S=repmat([1 .5]*8,prod(size(x)),1);
scatter3(xx(:),yy(:),zz(:),S(:),'k','filled')
set(gcf,'color','w')
xlabel('\bf{x}','fontsize',12)
ylabel('\bf{y}','fontsize',12)
zlabel('\bf{z}','fontsize',12)
title('\bf{The usage of ''scatter3.m'' function}','fontsize',12)
```

위와 같은 일련의 명령어를 실행하면, [그림 3-61]의 결과를 얻는다.



[그림 3-61] "scatter3.m"함수의 사용 방법

이제, 복소수를 표현해주는 함수들에 대해서 알아보자.
앞에서 배운 "**plot.m**" 따위의 함수들은 real part를 x 축의 좌표로, imaginary part를 y 축의 좌표로 하여 plotting 한다고 하였다.
그러나, 대부분의 plotting 함수들은 보통 허수 부(imaginary part)를 무시하고 그래프를 작성한다. 물론, 그래프를 작성하는 도중에 허수 부를 무시하는 경우 Matlab은 다음과 같은 경고문(Warning)을 내보낸다.

"Warning: Imaginary parts of complex X and/or Y arguments ignored."

본 단원에서는 복소수를 표현해주는 대표적인 함수로 "compass.m, feather.m, polar.m, rose.m"과 같은 함수들을 알아 볼 것이다.

본 단원에서 살펴볼 함수들은 방향을 원소로 하는 벡터(direction vectors)**와 속력을 원소로 하는 벡터**(velocity vectors)**로 구성된 데이터를** display**하는 경우에도 사용 될 수 있다. 즉, 어떠한, 물체의 움직임 추적을 할 때 사용될 수 있는 그래픽 함수들이다.**

카) compass.m : **제 1 부류**

"compass.m"함수는 주어진 복소수 데이터의 각도(angle)**와 크기**(magnitude)**를 원형 격자**(circular grid)**안에서 표현 해준다.**
"compass.m"함수는 주어진 좌표의 원점에서 발산하는(emanating) 화살표를 만들어서 복소수를 표현한다.
화살표(arrow)**의 길이는 각각의 복소수에 대한 크기를 나타내고, 화살표가 가리키는 방향**(arrow's pointing direction)**은 복소수의 각도를 의미한다.**

전형적인 사용 방법(syntax) : **compass.m**
a) **h=compass(X, Y)**
***real 벡터** X, Y**를** real(X)+imag(Y), **즉, 복소수로 간주하여** plotting**한다. 주어진 벡터** X, Y**의 크기는 같아야 한다.**
***화살표의 개수는 벡터** X**(또는,** Y**)가 갖고 있는 원소의 수와 같다.**
***화살표의 끝**(tip)**은 좌표의 원점과** [X(i), Y(i)]**를 이은 선**(line)**상에 있다.**
b) **h=compass(..., LineSpec)**
***화살표를 주어진 LinSpec에 의거하여 그린다.**
***Line Object에 해당하는** handle, **h를 돌려준다.**

만일, Z**가 복소수를 원소로 갖는 행렬이라면,** **h=compass(Z)의 표현은** **h=compass(real(Z), imag(Z))와 같은 표현으로 간주한다.**

[**예 제**] **벡터** Z**의 원소들은 크기가** [1:10]**이고, 각도가** [1:10]*36*(pi/180)**이다.**
벡터 Z**에 대한** 10**개의 원소들을 "compass.m"함수로 표현하여라.**

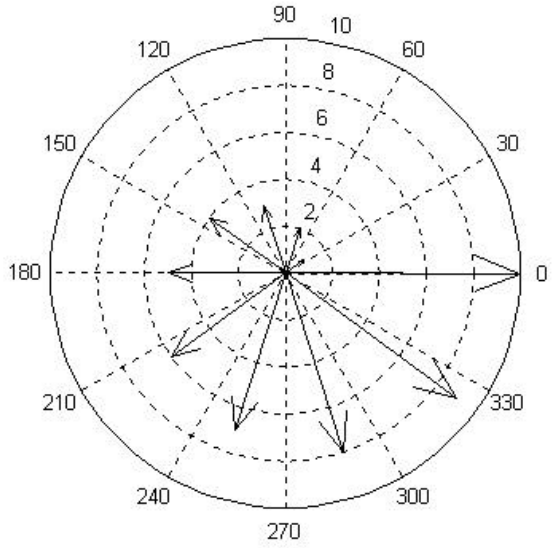
[**풀이**]

```
Z=[1:10].*exp(i*[1:10]*36*(pi/180));
h=compass(Z);
set(gcf,'color','w')
set(h,'color','k')
```

위와 같은 일련의 명령어를 실행하면, 벡터 Z**의 원소들인 복소수는** 10**개의 화살표로 표현된다.**

이들 화살표의 크기는 복소수 원소들의 크기를 표현하고, 화살표의 방향은 복소수 원소들의 각도를 나타낸다.

[그림 3-62]는 결과를 보여 주고 있다.



[그림 3-62] "compass.m"함수의 사용 방법

타) feather.m : 제 1 부류

간혹, "compass.m"함수처럼 모든 화살표가 공통의 원점(origin)을 갖는 것보다는 일정하게 떨어진 수평축을 따라서 복소수를 표현하는 화살표들이 나타나기를 원하는 경우가 있을 것이다. 이와 같은 경우에 사용되는 그래픽 함수가 "feather.m"함수이다.
사용방법은 "compass.m"함수와 같다.

전형적인 사용 방법(syntax) : **feather.m**

a) **h=feather(X, Y)**
*real 벡터 X, Y를 real(X)+imag(Y), 즉, 복소수로 간주하여 plotting한다. 주어진 벡터 X, Y의 크기는 같아야 한다.
*화살표의 개수는 벡터 X(또는, Y)가 갖고 있는 원소의 수와 같다.
*화살표들은 일정한 간격으로 떨어져있다.

b) **h=feather(..., LineSpec)**
*화살표를 주어진 LinSpec에 의거하여 그린다.
*Line Object에 해당하는 handle, h를 돌려준다.

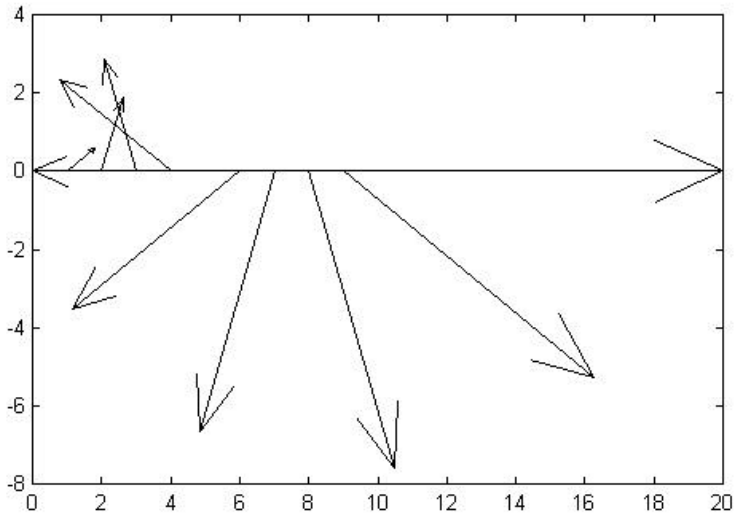
만일, Z가 복소수를 원소로 갖는 행렬이라면, h=feather(Z)의 표현은 h=feather(real(Z), imag(Z))와 같은 표현으로 간주한다.

[예 제] 앞에서 "**compass.m**"을 사용하여 살펴본 예제를 이번에는 "**feather.m**"함수를 이용하여 살펴보자.

[풀이]
"**feather.m**"함수를 사용하기 위하여 다음과 같은 명령을 실행하자.

```
Z=[1:10].*exp(i*[1:10]*36*(pi/180));
h=feather(Z);
set(gcf,'color','w')
set(h,'color','k')
```

위와 같은 일련의 명령어를 실행하면, [그림 3-63]과 같은 결과를 얻는다.



[그림 3-62] "feather.m"함수의 사용 방법

벡터 Z의 원소들인 10개의 복소수들이 화살표로 표현되어 등 간격으로 나타나고 있다. 각각의 화살표의 길이는 복소수의 크기(magnitude)를, 화살표와 수평축이 이루는 각은 복소수의 각도(angle)를 나타낸다.

지금까지 배운 모든 그래픽 함수들은 모두 직각 좌표 계(Cartesian coordinate system)를 이용하는 함수들이었다. 그러나, "**rho와 theta**"로 데이터를 표현하는 극 좌표 계(polar coordinate system)를 이용하여 데이터를 표현할 수도 있다. 이와 같이 주어진 데이터를 극 좌표 계에서 표현해주는 함수로는 "**polar.m, rose.m**"함수들이 있다.

파) polar.m : **제 1 부류**

주어진 데이터를 대응하는 각도(theta, 단위는 radian)와 크기(rho)로 변환해주면, "polar.m"을 이용하여 극 좌표 계로 데이터를 표현해 줄 수 있다.

전형적인 사용방법(syntax) : **polar.m**

a) **h=polar(theta, rho)**

*****각도를 나타내주는 배열 theta와 반지름을 나타내는 배열 rho 을 가지고, polar coordinate plot을 작성한다.

*****Theta는 x 축과 반지름 벡터간의 각도(단위는 radian)를 나타내 준다.

b) **h=polar(theta, rho, LineSpec)**

*****polar plot을 구성하는 Lines의 style을 결정한다.

*****Line Object에 해당하는 handle, **h**를 반환한다.

만일, theta와 rho가 행렬이면, theta의 열 벡터들은 rho의 열 벡터에 대해서 plotting 된다. 앞에서, 신호처리에서 사용되는 데이터 행렬은 항상 열 벡터의 단위로 계산한다고 하였다. 즉, 각각의 신호들은 행렬의 열 벡터들에 해당한다.

Matlab은 **거의 대부분의 함수들이 column-wise 연산을 행한다.**

[**예 제**] **각도가** 2*pi*[0:.01:1]**의 범위를 갖고,** rho=0.5+cos(theta)**로 정의된다고 하자.** theta**와** rho**를 극 좌표 계로** plotting**하여라.**

[**풀이**]

다음과 같은 commands**를 생각 할 수 있다.**

□**theta=2*pi*[0:.01:1];**
□**rho=0.5+cos(theta);**
□**polar(theta,rho,'k')**
□**set(gcf,'color','w')**

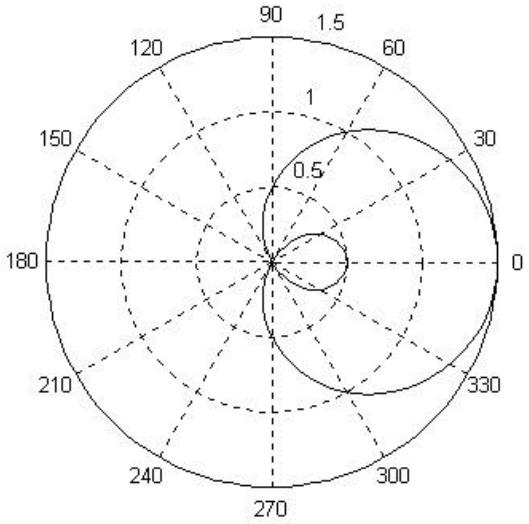
위와 같은 일련의 명령어들을 사용하면, [**그림** 3-63]**의 결과를 얻을 수 있다.**


주의

만일, theta와 rho의 범위(range)**를 바꾸기 위해서, 앞에서 배운** axis([min_theta max_theta min_rho max_rho])**와 같은 명령을 사용하면 안 된다.**
왜냐하면, "polar.m"함수는 "plot.m"함수를 이용하여 만든 것이기 때문이다.
즉, "polar.m"함수는 "plot.m"함수를 이용하여 그래프를 만들고, "axis off" 명령어로 좌표축을 없앤 것이다. 그러므로, "axis on"**이라고 명령하면, 사라졌던 좌표축이 다시 생긴다.**

Matlab Command	
axis on	**좌표축을** screen**상에** display**한다**.
axis off	**좌표축을** screen**에서 지운다**.

"**axis on/off**"**명령어는 앞에서 배운** Axes Object**의 "Box on/off"명령과 같은 기능을 한다**



[**그림** 3-63] "polar.m"**함수의 사용 방법**

하) rose.m : 제 1 부류

"**rose.m**"함수는 20개이하의 **bins**로 **theta**의 분포를 표현해주는 함수이다.
개개의 **bins**는 주어진 각도의 범위들 안에 포함된 theta의 개수를 의미한다.
결국, "**rose.m**"함수로 각도(**theta**)들에 대한 **Histogram**을 만들 수 있다.

전형적인 사용 방법(syntax) : **rose.m**

a) **h=rose(theta)**
***theta**에 대한 각도 histogram을 만들어 준다.
b) **h=rose(theta, x)**
*length(x)는 만들어질 bins의 수를 의미하고,
벡터 x의 값들은 각각의 bins에 대한 중심각을 의미한다.
c) **[tout, rout]=rose(...)**
***tout**과 **rout**벡터를 반환하여, **polar(tout, rout)**로부터 각도에 대한 histogram을 얻는다.
d) **h=rose(theta, nbins)**
*[0 2*pi]범위 안에서 균등하게 떨어진 nbins를 만들어
각도에 대한 histogram을 얻는다.

[예 제] 예를 들어서, 12시간 동안 매시간 마다 바람이 부는 방향을 측정하여 변수 **wdir**에 저장하였다고 하자.

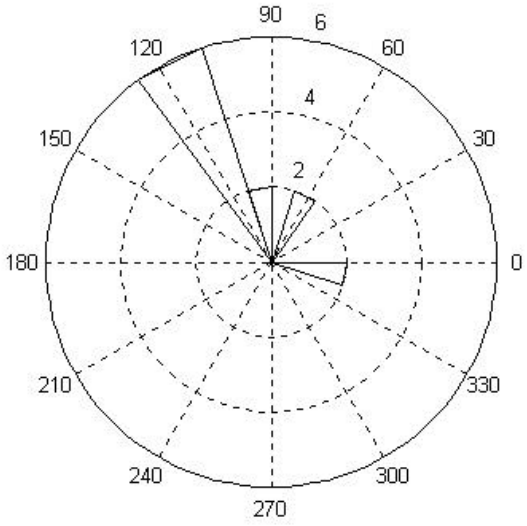
```
wdir=[45 90 90 45 360 335 360 270 335 270 335 335];
```

데이터 **wdir**에 근거하여 바람이 부는 방향(각도)에 대한 histogram을 작성하여라.

[풀이]
우선, 주어진 데이터 **wdir**을 radians로 바꾸고, "**rose.m**"함수의 입력 변수로 넣는다.

```
□wdir=wdir*pi/180;
□h=rose(wdir)
□set(gcf,'color','w')
□set(gcf,'color','w')
□set(h,'color','k')
```

위와 같은 일련의 명령어를 실행하면, [그림 3-64]와 같은 결과를 얻는다.



[그림 3-64] "rose.m"함수의 사용 방법

[그림 3-64]로부터 바람이 12시간 동안 주로 335(degree)로 불었다는 것을 알 수 있다.



주의 책에는 제 3 부류에 속하는 "pie.m, pie3.m, bar.m, barh.m, bar3.m, bar3h.m, area.m, errobar.m, findobj.m ezplot.m"함수가 이 부분에서 설명된다. 단, "ezplot.m"함수는 [부록]에 실은 "symbolic math.의 사용법"에서 설명할 것이다.

제 3부류에 속하는 함수들을 설명하는 데, 그림이 23개, 표가 1개 사용되었다.

IV. Animation :

Matlab에서 Animation이란 object를 생성하고, object의 위치를 변화시키면서 재차 생성하는 과정을 말한다.
대표적으로 demo 함수인 "**lorenz.m**"을 실행시키면, Matlab에서 의미하는 animation 효과를 볼 수 있다.
Matlab에서의 animation은 "**EraseMode**" propertyname에 의해 여러 가지 다른 효과를 줄 수 있다.
"EraseMode" propertyname은 주어진 object를 "**언제, 어떻게**" 생성하고, 지울 것인지를 결정한다.
그런데, "EraseMode" propertyname은 모든 object에 존재하는 것은 아니다. 단지, "**lines, patches, surfaces, text**" objects에만 존재한다.

[표 3-13]에는 "**EraseMode**"에 대한 propertyvalue를 설명하고 있다.

"**EraseMode**" propertyname	
propertyvalue	각각의 propertyvalue에 대한 설명
normal	default propertyvalue이다. object가 갱신 될 때마다 Axes를 refresh한다. 그러므로, object에 변화가 있을 때마다 **flicker**가 발생한다.
background	Figure의 배경색과 같은 색으로 object를 바꾸어서 지우는 효과를 준다. 결국, 지워진 object의 데이터에 손상이 발생한다.
xor	Figure의 배경색과 object의 색에 대해 "**xor**"를 행하여 object를 지운다. 그러므로, 지워진 object의 데이터에 손실은 발생하지 않는다.
none	object의 변화가 있을 때, 이전 object를 지우지 않는다.

[표 3-13] "EraseMode" propertyname에 대한 설명

[표3-13]을 정리하면 다음과 같다.

(1) "**flicker**"가 발생하지 않는 animation을 만들고 싶으면, "**normal**"을 사용하지 말아라.
(2) "**lorenz.m**"함수를 실행시켜 보면, object의 변화에 따른 꼬리(trail)가 존재한다.
이와 같은 효과를 주려면 "**none**"을 이용하여라.
(3) 새로 생성되는 object에 의한 animation에만 관심이 있다면, "**background**"를 이용하여라.
(4) object의 데이터 손상을 원하지 않으면, "**xor**"를 이용하여라.

"**fig3_65.m**"는 "**y=sinc(w*x)**"에서 "**w**"의 값을 바꾸어 가면서 그래프의 변화를 살펴보는 M-file이다.

fig3_65.m

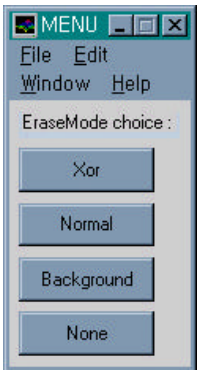
```
%
%Animation에 대한 예제 : Ani.m
%
%1998.2.21          By Lim Jong_Su
%

close all
clear all

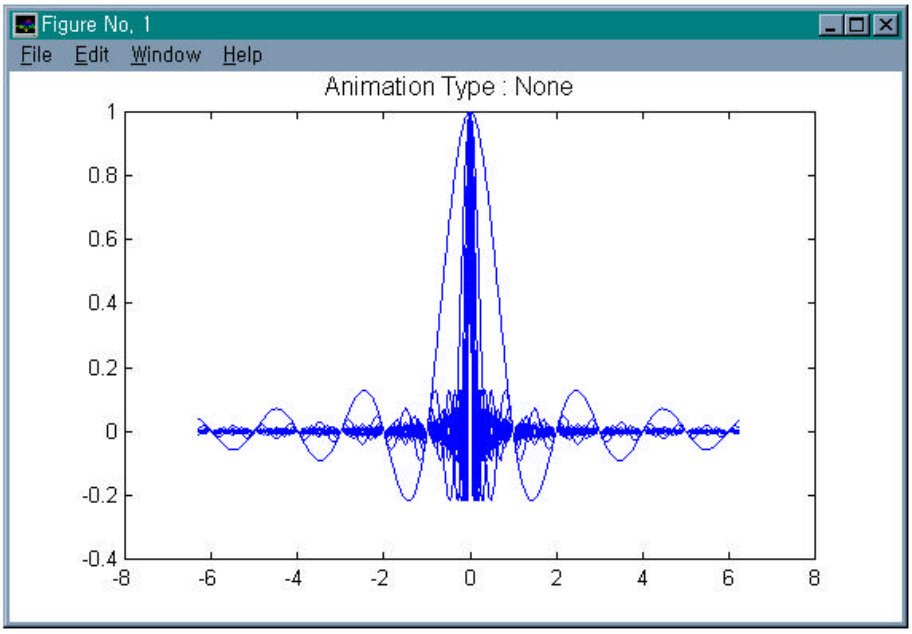
w=1;
x=-2*pi:1/100:2*pi;
y=sinc(w*x);
K=MENU('EraseMode choice :','Xor','Normal','Background','None');
mode=str2mat('Xor','Normal','Background','None');
fig_size =[121    208    560    346];
figure('pos',fig_size);
h=plot(x,y,'erasemode',mode(K,:));
set(gcf,'color','w')
title(['Animation Type : ' mode(K,:)],'fontsize',12);
for index=1:2:20
    y=sinc(index*x);
    set(h,'ydata',y)
    drawnow   %Figure window를 update하는 함수.
end
```

앞에서 배운, "**menu.m**"함수를 이용하여 "EraseMode"를 선택하게 하였다.

[그림 3-66(a)]는 "**Ani.m**"함수를 실행 시켰을 때, 발생하는 "**menu**" window이다.
이제, "Menu" window에서 원하는 Mode를 선택하면, Matlab이 지원하는 4가지 animation mode를 볼 수 있을 것이다.
[그림 3-66(b)]는 "**None**"를 선택한 경우의 결과이다.



[그림 3-66(a)]



[그림 3-66(b)] "EraseMode"가 "None"인 경우

Matlab에서는 보다 편리하게, animation을 사용 할 수 있도록 "**comet.m, comet3.m**"함수를 제공한다.

"**comet.m**"함수는 하나의 작은 원(circle, comet함수의 **head**라고 부른다.)이 주어진 데이터를 그려나가는 함수이다. 또한, "**comet.m**"함수는 head가 그래프를 그려나갈 때, 긴 꼬리(trail, comet함수의 **body**라고 부른다.)를 갖는다.

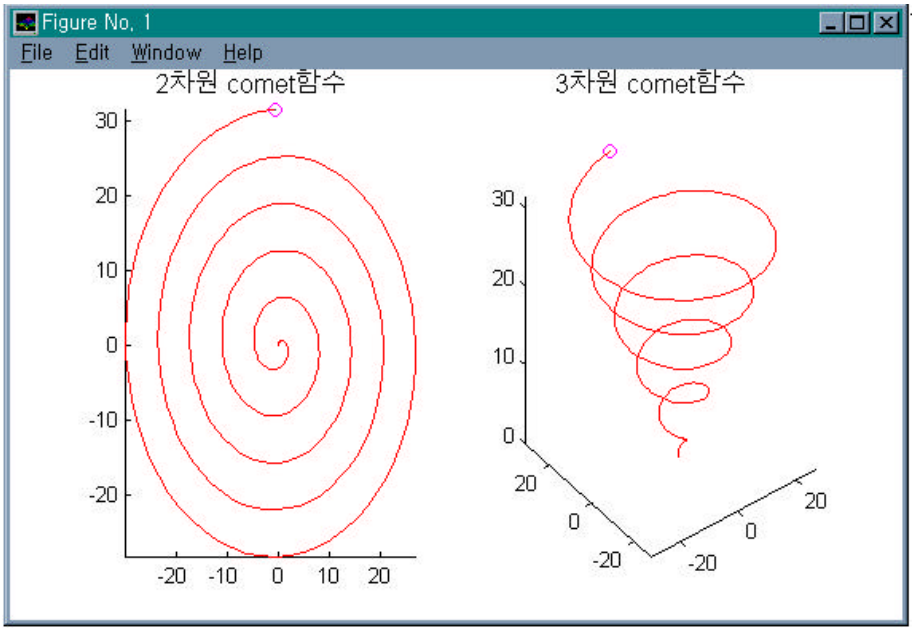
전형적인 사용 방법(Syntax) : **comet.m**
comet(x, y)
→벡터 x에 대한 벡터 y값을 그려나간다.

전형적인 사용 방법(Syntax) : **comet3.m**
comet3(x, y, z)
→[x(i), y(i), z(i)] 점들을 통하여 하나의 곡선을 그려나간다.

[그림 3-67]는 "**comet.m, comet3.m**"함수들에 대한 사용 예제를 보여 주고 있다.



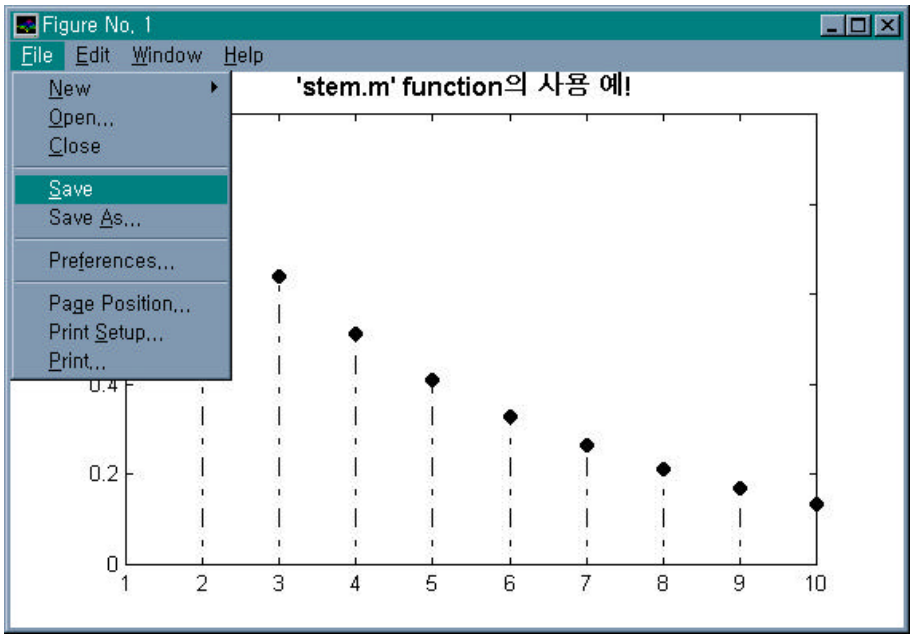
"**EraseMode**"가 "**none**"인 경우, figure의 크기를 재조정하면, 가장 최근에 생성된 (또는, 그려진) object만 남고 이전에 그려진 object는 모두 소멸된다. 즉, 이전 데이터는 figure window내에서만 보일 뿐이다. 그러므로, "**copy, print**" 명령을 실행하면 최종 **object**만 나타난다. [그림 3-67]은 screen capture로 얻은 것이다.



[그림 3-67] "comet.m, comet3.m"함수들의 사용법

V. Figures는 어떻게 저장되는 가?

지금까지 여러분은 많은 그래픽 tool을 배웠다. 대부분의 경우, 생성한 그래픽을 나중에 다시 보기 위하여 해당 M-file을 저장한다.
그러나, 해당 M-file을 저장하지 안고도 다시 그래프를 생성하는 방법이 있다.
이 방법은 GUI 프로그램에서도 적용되는 방법이다.
예를 들어서, [그림 3-57]을 생성 한 경우, window 화면 내의 "**File**" menu를 선택하면 [그림 3-68]과 같은 결과가 나온다.



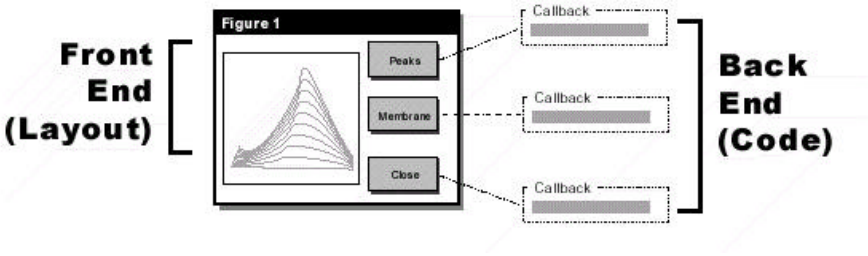
[그림 3-68] Figure에 save의 의미

이제, "**Save**"를 선택하면, 저장할 file 이름을 물어 볼 것이다. 알맞은 이름을 정하여 저장해 본다. 여기서는, "**fig3_68.m**"이라고 하였다.
그러면, 해당 디렉토리에 "**fig3_68.m**"과 "**fig3_68.mat**"라는 file들이 생긴다.
여기서, 중요한 것은 "**fig3_68.m**"과 "**fig3_68.mat**"라는 file들이 갖고 있는 데이터는, 해당 그래프에 있어서 "**Front-End(Layout)**"에 대한 것뿐이다.
"**Front-End(Layout)**"은 주어진 그래프에서 눈에 보이는 부분만을 의미한다.

만일, 여러분이 GUI 프로그램을 작성하여 "Save"명령어로 저장한 경우에도 동일하게 해당 그래프에 대한 "***.m, *.mat**"파일들이 생성된다. 그러나, 해당 M-file을 실행시키면 겉모습은 동일하게 생성되지만, 해당 Object를 click한 경우, GUI 프로그램이 작동하지 않는 것을 볼 수 있을 것이다.
이것은 모든 Matlab 그래프를 design하는 데는 2 가지 방법이 있기 때문이다.
(1) 눈으로 보이는 부분을 디자인하는 "**Front_End layout**" part.
(2) 실제로 해당 Object에 대한 함수 기능을 디자인하는 "**Coded Back_End**" part.

물론, 지금까지 설명하여 온 그래픽 함수들은 특별히 기능이 부여되지 않았으므로 "**Back_End(Code)**"부분은 생각하지 않아도 된다.
[부록]에 실은 "**GUI 프로그램 작성법**"에서는 "**Back_End(Code)**"부분에 대해서 알 아 볼 것이다.

[그림 3-69]는 이들 "**Front_End(Layout)**"part와 "**Back_End(Code)**"part에 대해서 설명하고 있다.



[그림 3-69] Front_End와 Back_End의 비교

일반적으로 "***.m**"파일은 해당 figure를 재생산하는 M-file이고, "***.mat**"파일은 text file에는 저장되기에 부적합한, 해당 figure 생성에 필요한 데이터 파일이다.
즉, "***.mat**" 파일에는 "**colormap**"과 "**Userdata**" 따위가 저장된다.

이제, "□close all ↲"명령을 실행하여 모든 그림들을 command window 내에서 지운다. 그리고, "**fig3_68.m**"파일을 실행시키면 [그림 3-57]이 생성된다. 그런데, "title" 부분이 만들어지지 않을 것이다. 이것은 "title" 부분에 있는 "**한글**" 때문이다.
여러분이 "**guide.m**"함수나 또는, 지금처럼 그래프를 저장 할 때, Matlab은 해당 그래프에 대한 M-file을 만들어 준다. 그러나, 한글이 그래프에 있으면, **"한글"이 포함된 열은 cancel**된다.

"**fig3_68.m**"파일의 내용 중에서 "한글"이 있는 부분은 글씨가 깨진 것을 볼 수 있을 것이다. 이제, "한글"이 깨진 부분을 다시 "한글"로 typing하면 "한글"이 출력 된 [그림 3-57]과 같은 화면을 얻을 수 있다.

5. 파일 입·출력

이번 단원에서는 원하는 임의의 파일을 Matlab에 읽고 쓰는 방법에 대해서 알아볼 것이다.

I. 파일의 입·출력과 관련된 함수들

Matlab이 제공하는 파일 입·출력 함수들은 임의의 이진(**arbitrary binary**)파일과 형식화된(formatted) ASCII 파일을 읽고, 쓸 수 있는 여러 종류의 함수들을 제공하고 있다.
이들 함수들은 여러 가지 형식(format)으로 설정된 데이터를 읽어들이고, 또한, 다른 프로그램이나 장비에 데이터를 기록하게 해준다.

본 단원에서 살펴볼 파일 입·출력 함수들은 2가지로 분류 할 것이다.

1. 일반적으로 많이 사용되는 "save.m, load.m"함수들에 대해서 알아본다.

이들 함수들은 다른 파일 입·출력 함수들에 비해 사용하기가 매우 용이하고, 또한, Matlab에서 데이터를 저장하고 복원할 수 있도록 제작된 최적화 된 함수들이다.

2. Low_level file I/O 함수들에 대해서 알아 볼 것이다.

Matlab이 제공하는 Low_level I/O 함수들의 이름은 C언어에서 제공되는 함수들과 이름이 동일하다.
그렇다면, Matlab이 제공하는 함수들과 C언어에서 제공하는 함수들간에는 어떠한 관계가 있을 가?
사실, Matlab이 제공하는 Low_level I/O함수들은 같은 이름의 C언어 입·출력 함수와 거의 동일한 기능을 한다. 여기서, "거의"라는 용어를 사용한 이유는 Matlab이 모든 피 연산자를 배열로 취급하는 반면에, C언어는 그렇지 않다는 데 있다.
즉, 기본적인 연산 단위가 Matlab에서는 배열이라는 점만 다르고, 나머지 기능은 C언어의 파일 입·출력 함수들과 동일하다고 보면 된다.

가. Save.m, Load.m :

보통, workspace에 있는 데이터들을 저장하는 가장 손쉬운 방법은 "save.m, load.m"함수들을 이용하는 것이다.
이제, 이들 함수들에 대해서 간략히 알아보자.

전형적인 사용 방법(syntax) : **save.m**
a) **save filename**
*모든 base workspace에 있는 데이터들을 **filename**에 저장한다.
b) **save filename 변수들**
*주어진 변수들만 **filename**에 저장한다.
c) **save filename 변수들 option**
***default format**인 **binary**파일이 아닌, **ASCII**형식으로 주어진 변수들만 저장하고 싶을 때, **option**을 이용한다.
option에 대해서는 [표5-1]참고.

사실, "save.m, load.m"함수들을 이용하여 데이터를 저장할 때는 보통, 확장자로 ".mat"을 이용하는 것이 좋다. 또한, ASCII형식으로 저장하는 것보다는 Binary형식으로 저장하는 것이 데이터 손실도 없고 안정적이다.
만일, 복소수를 ASCII형식으로 저장한다면, 허수 부는 손실된다는 점에 유의하라!
그러므로, 필자가 권하고 싶은 데이터 파일 형식은 "확장자가 mat인 Binary"파일로 데이터를 저장할 것을 권한다.

"save.m"함수로 데이터를 저장하여 만들어진, MAT-files는 "double-precision binary MATLAB format"이다. 이들 mat-files는 다른 시스템에서도 Matlab에 의하여 쉽게 읽혀질 수 있다. 그러나, 시스템간의 차이에 의해서 발생하는 기계적 오차(즉, eps)는 감수해야 한다.

[표 5-1]은 "save.m"함수에서 필요한 option을 정리한 것이다.

options	option에 따라 저장되는 형식
-ascii	8-digit ASCII format으로 저장한다.
-ascii -double	16-digit ASCII format으로 저장한다.
-ascii -tabs	tab에 의해서 분리된 데이터를 8-digit ASCII 형식으로 저장한다..
-ascii -double -tabs	tab에 의해서 분리된 데이터를 16-digit ASCII 형식으로 저장한다..

[표 5-1] "save.m"함수에서 사용되는 options

"save.m"함수에 의해서 저장된 데이터 파일을 복원하는 데, 사용되는 함수로는 "load.m"이라는 함수가 있다.

그 사용 방법은 다음과 같다.

전형적인 사용방법(syntax) : **load.m**
a) **load filename**
***주어진 filename에 저장된 데이터들을 복원(retrieve)한다.**
b) **load filename -ascii**
***ASCII형식으로 저장된 filename을 복원한다.**

이제, 예제를 통하여, "**save.m, load.m**"함수의 사용방법을 익혀 보도록 하자.

```
□x=magic(3)
□y='Lim Jong Su';
□z=2+3i;
□whos
Name      Size          Bytes  Class

  x         3x3            72  double array
  y         1x11           22  char array
  z         1x1            16  double array (complex)

Grand total is 21 elements using 110 bytes
```

위와 같이 우선, "**double**"형과 "**char**"형의 데이터를 만들어 보자.
이제. 이들 두 가지 종류의 데이터를 확장자가 **mat**인 **Binary**파일로 저장하겠다.

```
□save temp_data.mat x y z  %"save temp_data.mat"라고 해도 된다.
□clear all                 %"ls"명령을 실행하면, "temp_data.mat"파일이
□whos                      %새로 만들어 진 것을 볼 수 있을 것이다.
□load temp_data.mat        %"clear all"명령으로 모든 데이터가 지워졌다.
□whos
Name      Size          Bytes  Class

  x         3x3            72  double array
  y         1x11           22  char array
  z         1x1            16  double array (complex)

Grand total is 21 elements using 110 bytes

□z
z =

   2.0000 + 3.0000i
```

결국, 데이터 x, y, z가 모두 복원되었다. 게다가, 복소수도 정확하게 복원되었다.
위에서, "save temp_data.mat"라고 해도 되는 이유는 현재 base workspace에 변수들은 모두 x, y, z뿐이기 때문이다.

확장자가 mat가 아닌 다른 확장자 명을 사용하고 싶다면 다음과 같이 하면 된다.
그러나, 확장자가 mat가 아닌 다른 파일들은 load할 때, 맨 뒤에 -mat라고 붙여 주어야 한다. -mat라고 붙여 주지 않으면, 주어진 파일을 ASCII text 파일로 간주하기 때문이다.

```
□save temp_data.dat x y z
□clear all
□load temp_data.dat -mat
□whos
Name      Size          Bytes  Class

  x         3x3            72  double array
  y         1x11           22  char array
  z         1x1            16  double array (complex)

Grand total is 21 elements using 110 bytes
```

위와 같이 본래 데이터들이 복원되었다.

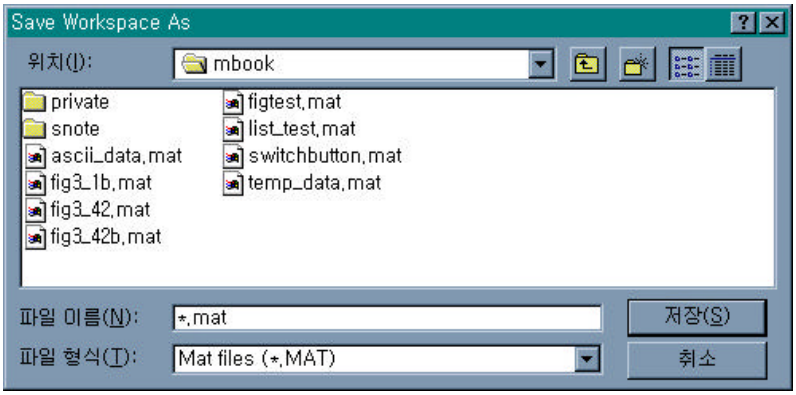
지금까지 설명한 "save.m, load.m"함수의 사용법은 사실, 더욱 간편하게 이용하는 방법이 있다.
Matlab의 command window의 File menu에 있는 "Save workspace as, Load workspace"를 사용하면 된다.

[그림 5-1]은 command window에 있는 File menu를 선택 한 경우에 나타나는 window이다.



[그림 5-1] "File" menu

이제, base workspace에 있는 데이터들을 "limdata.mat"라는 파일에 저장하고 싶다고 하자. base workspace에 있는 데이터들을 저장하기 위해서는 우선, "Save Workspace As..."을 선택한다. 그러면, [그림 5-2]와 같은 window가 나타날 것이다.

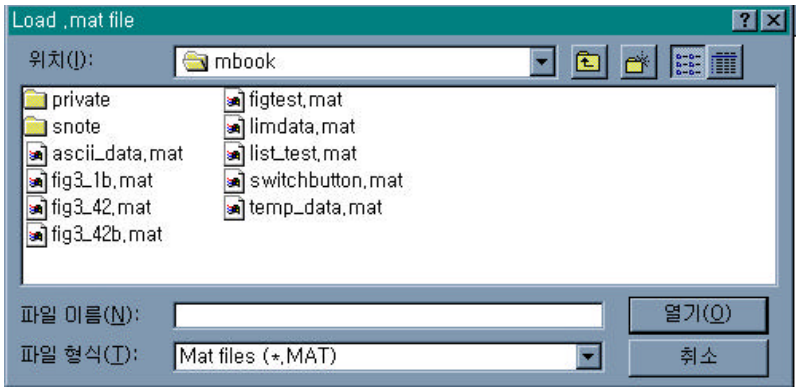


[그림 5-2] "Save Workspace As" window

이제, "파일 이름(N)"에 있는 "*.mat"에 "limdata.mat"라고 typing하면, "mbook"디렉토리에 새로운 파일인 "limdata.mat"라는 파일이 첨가된다.

여기서, 주의 할 것은 "limdata.mat"라는 파일에 저장되는 데이터들은 base workspace에 있는 변수들 모두라는 것이다. 그러므로, 원하는 변수들만 선택하여 저장하는 데는 부적합하다.

어쨌든. "limdata.mat"에 저장된 데이터를 복원하려면, [그림 5-1]에 있는 "Load Workspace"를 선택하면 된다.
만일, "Load Workspace"를 선택하였다면, [그림 5-3]을 얻을 수 있을 것이다.



[그림 5-3] "Load.mat file" window

이제, 복원하려는 데이터를 갖고 있는 파일이름을 "파일 이름(N)"에 typing하고, "열기(O)" 버튼을 누르면, 원하는 데이터들이 base workspace에 첨가된다.

나. Low_level file I/O 함수들 :

사용하는 언어에 관계없이 주어진 파일을 읽고 쓰려면 제일 먼저 주어진 파일을 열어야 한다. 물론, 파일에 일고 쓰는 작업이 모두 끝났으면, 파일을 닫아 주어야 할 것이다.
이와 같은 기능을 하는 함수로는 "fopen.m, fclose.m"과 같은 함수들이 있다.
이들 함수의 이름이 꽤 눈에 익을 것이다. 바로, C언어에서도 같은 이름의 함수들이 있기 때문이다.

우선, "fopen.m"함수의 사용법을 간단히 알아보자.

전형적인 사용 방법(Syntax) : **fopen.m**
a) **fid=fopen(파일 이름, 모드(mode))**
*주어진 파일이름(확장자 포함)을 여는데(open) 성공하면,
파일 식별자(file identifier), **fid**에 **3**이상의 양의 정수를 돌려준다.
***fid**는 다른 파일 I/O함수들에 첫 번째 입력 매개변수로 사용된다.
*그러나, 주어진 파일이름(확장자 포함)을 여는데(open) 실패하면,
파일 식별자(file identifier), **fid**에 "-1"값을 돌려준다.
b) **[fid, message]=fopen(파일이름 , mode)**
*만일, 주어진 파일을 읽는데 실패하면, **message**라는 변수에 실패한
이유가 문자열로 반환된다.

보통 Matlab 매뉴얼을 보면, 필자가 사용하는 **mode**라는 단어 대신에 **permission**이라는 단어를 사용하고 있다.

그런데, C언어에서는 permission이라는 단어보다는 mode라는 단어를 즐겨 사용한다. 그래서, 독자들로 하여금 이해를 돕기 위해서 permission이라는 단어 대신에 친숙한 mode라는 단어를 사용하고 있는 것이다.

또한, MS-Window, VMS system은 text파일과 binary파일을 읽어 들일 때, mode에 부가적인 문자를 붙여 주어야한다. 즉, text파일을 일어들일 때는 mode뒤에 "t"를 덧붙여 주어야 한다.
binary파일은 default mode이므로 "b"라고 덧붙일 필요는 없다.

그러나, Unix, Mac. system은 text파일과 binary파일의 구별이 없으므로 덧붙이는 작업은 하지 안아도 된다.

[표 5-2]는 사용할 수 있는 mode의 종류를 나타낸 것이다.

mode	mode에 따른 설명
'r'	default mode이고, 읽기(reading) 위해서 주어진 파일을 연다.
'r+'	읽고, 쓰기 위해서 주어진 파일을 연다.
'w'	주어진 파일 이름을 갖는 새로운 파일을 만든다. 그러나, 주어진 파일 이름과 같은 파일이 존재하면, 지우고(delete) 새로 파일을 만든다. 새롭게 만들어진 파일을 쓰기 위해서 연다.(open)
'w+'	주어진 파일 이름을 갖는 새로운 파일을 만든다. 그러나, 주어진 파일 이름과 같은 파일이 존재하면, 지우고(delete) 새롭게 파일을 만든다. 만들어진 파일을 일고, 쓰기 위해서 연다.(open)
'a'	새로운 파일을 만들고 연다(open). 그러나, 같은 이름의 파일이 존재하면, 쓰기 위해서 파일을 연다. 그리고, 파일의 끝에 데이터를 첨가해 나갈 수 있도록 해준다.(appending)
'a+'	새로운 파일을 만들고 연다(open). 그러나, 같은 이름의 파일이 존재하면, 읽고, 쓰기 위해서 파일을 연다. 그리고, 파일의 끝에 데이터를 첨가해 나갈 수 있도록 해준다.(appending)

[표 5-2] mode의 종류와 설명

이제, 열려있는 파일을 닫아주는 함수인 "fclose.m"에 대해서도 간단히 알아보자.

전형적인 사용 방법(Syntax) : **fclose.m**

a) **st=fclose(fid)**
*"**fopen.m**"함수에 의해서 반환된 파일 식별자(file identifier)를
사용하여 열려있는 파일을 닫아 준다.
*파일을 닫아 주는 데 성공하면, "0"을 돌려주고,
실패하면 "-1"을 반환한다.

b) **st=fclose('all')**
*열려 있는 모든 파일을 닫아 준다.

이제, 여러분은 파일을 열고 닫을 수 있는 방법을 배웠다.

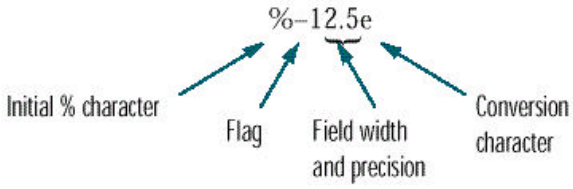
이번에는 데이터 파일을 읽고, 쓰는 방법에 대해서 배워보자.
여기서는 가장 많이 사용되는 두 개의 함수들, 즉, "fprintf.m, fscanf.m"함수들에 대해서만 알아 볼 것이다.

전형적인 사용 방법(Syntax) : **fscanf.m**

a) **count=fprintf(fid, format, A, ...)**
***행렬 A에서 실수 부(real part)를 주어진 format에 맞추어 열려 있는 파일(fid)에 쓰기(write)를 행한다.**
***쓰여진 bytes수는 count변수에 반환된다.**

b) **fprintf(format, A, ...)**
***screen(standard output)에 행렬 A를 표시한다.**

"fscanf.m"함수에서 언급한 "format"의 의미는 [그림 5-4]를 보면 알 수 있을 것이다.



[그림 5-4] format의 의미

즉, 주어진 배열 A를 지수형태로 변환하여, 정수부는 12자 이내로 하고, 소수부는 소수점이하 5자리까지 주어진 field안에서 왼쪽으로 맞추라는 뜻이다.

[표 5-3]은 이들 format의 형태 중에서 Escape 문자에 대한 것이다.

Escape 문자	Escape 문자에 대한 설명
\n	New Line
\t	Horizontal tab
\b	Backspace
\r	Carriage return
\f	Form feed
\\	Backslash를 표시
\'' 또는 ''	단일 인용 부호를 표시
%%	percent 문자를 표시

[표 5-3] "Escape" 문자들의 종류

또한, [표 5-4]는 Conversion Specifiers에 대한 것이다.

Conversion Specifier	각각에 대한 설명
%c	하나의 문자로 표시
%d	십진법으로 표시(signed)
%e	지수 형태로 표시(소문자 e를 이용)
%E	지수 형태로 표시(대문자 E를 이용)
%f	고정 소수점으로 표시
%g	중요하지 않은 zeros는 표시하지 안는다. 결국, %e와 %f보다 간결한 표현
%o	8진법으로 표시(unsigned)
%s	문자열로 표시
%u	십진법으로 표시(unsigned)
%x	16진법으로 표시(소문자 a f를 이용)
%X	16진법으로 표시(대문자 A F를 이용)

[표 5-4] Conversion Specifiers의 종류

이번에는 나머지 format에 관련된 문자들에 대해서 알아보자.
[표 5-5]를 참고하기 바란다.

format에 관련된 문자들	설명	사용 예
A minus sign(즉, -)	주어진 field의 왼쪽으로 정렬	%-5.2d
A plus sign(즉, +)	부호를 표시하게 한다. (즉, +, -)	%+5.2d
zero(즉, 0)	주어진 field의 spaces에 zeros를 채움	%05.2d

[표 5-5] format에 관련된 문자들

다음은 데이터를 읽어들이는 함수인 "fscanf.m"에 대해서 알아보자.

전형적인 사용방법(Syntax) : **fscanf.m**

a) **[A, count]=fscanf(fid, format, size)**
*"fopen.m"함수에 의해서 열려있는 파일(fid)을 주어진
format에 의거하여 size에 상술된 크기로 읽어 들인다.
*count는 읽어들인 데이터의 수를 의미한다.

b) **A=fscanf(fid, format)**
*format에 의거하여 데이터를 읽어 들인 뒤에 열 벡터의 형태로
반환한다.

"fscanf.m"에서 언급한 size는 3가지 형태를 취할 수 있다.
다음에 나오는 내용을 보고 size가 취할 수 있는 형태에 대해서 익혀두자.

n	n개의 데이터를 열 벡터의 형태로 읽어 들인다.
inf	주어진 파일의 끝까지 읽어 들인다.
[m, n]	m×n의 크기를 갖는 행렬의 형태로 읽어 들인다. 여기서, n은 inf가 될 수 있으나, m은 될 수 없다.

이제, 여러분은 데이터를 저장하고, 읽어들이는 함수들에 대하여 학습하였다.

이번에는 배운 함수들을 사용하는 방법에 대해서 알아보자.

```
□X=0:.1:1;
□y=[X ; exp(X)];
□fid=fopen('exp.txt','w')        %쓰기 위해서 "exp.txt"파일을 연다.(open)

fid =

     3

□count=fprintf(fid, '%6.2f  %12.8f\n', y)

count =

   220

□fclose('all');
```

위와 같이 명령하면, "exp.txt"라는 text file이 만들어진다. 그런데, 파일 식별자(fid)의 값이 "1"이 아니라, "3"이 나왔다.
그 이유는 파일 식별자가 "1"인 경에는 screen에 표시하라는 뜻이 되고, "2"인 경우에는 표준 error를 나타낸다. 그래서, 모든 파일들은 "3"부터 파일 식별자에 할당되는 것이다.

이제, "exp.txt"파일을 열어서, 데이터를 읽어보자.

```
□fid=fopen('exp.txt','r')        %읽기 위해서 "exp.txt"파일을 연다.(open)
□[a, count]=fscanf(fid, '%f  %f\n',[2 inf]);
□a'

ans =

         0    1.0000
    0.1000    1.1052
    0.2000    1.2214
    0.3000    1.3499
    0.4000    1.4918
    0.5000    1.6487
```

```
0.6000    1.8221
0.7000    2.0138
0.8000    2.2255
0.9000    2.4596
1.0000    2.7183
```

□**fclose('all');**

지금까지, 여러분은 데이터를 파일에 저장하고 읽어 들이는 방법을 배웠다.

%%%%%%%%%%%%%%%%%% **THE END** %%%%%%%%%%%%%%%%%%